2. Linear regression and optimization

Learn a numerical prediction rule from its errors

Preparation. Session 1's experiment protocol, arrays, averages, and a derivative as a local rate of change. Matrix calculus is not required.

Learning goals. Interpret slopes with units; calculate residuals and squared loss; fit a line with a formula and gradient descent; distinguish optimization from generalization; compare a predictor against a training-mean baseline.

120-minute route. Model and loss (20 min); least squares (20); multiple features and gradients (30); notebook and diagnostics (30); closing exercise (20).

1. Predict a numerical quantity

Let s be payload size in MB and y latency in milliseconds. A simple linear predictor is

$$\widehat{y} = b + ws, \qquad r = y - \widehat{y}.$$

The intercept b is predicted latency at zero payload. The slope w is the change in predicted latency per additional MB. If zero is outside the observed range, the intercept remains mathematically useful but its physical interpretation needs care.

A line predicts an average relationship. It does not assert that every request with the same payload takes exactly the same time. Queue length, unrecorded factors, and noise can produce different outcomes at the same input. A fitted slope also does not prove a causal effect: observational associations can reflect other variables.

Choose a line by defining an error objective

Positive and negative residuals can cancel. Squared residuals cannot:

$$\mathrm{MSE}(b, w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - b - ws_i)^2.$$

The units are ms squared. Squaring penalizes a large miss more strongly than several small misses with the same total absolute error. Outliers can therefore strongly influence a least-squares fit.

For observed values $(10, 20)$ and predictions $(12, 17)$, residuals are $(-2, 3)$. Their mean is 0.5, but MSE is $(4 + 9)/2 = 6.5$ and RMSE is $\sqrt{6.5} \approx 2.55$ ms. The small signed mean conceals both prediction errors.

A learning algorithm chooses parameters that reduce the objective on training observations. It does not have access to future outcomes. A good training fit is therefore evidence about optimization, not enough evidence about usefulness.

Check your understanding. Models A and B have residuals $(0, 0, 6)$ and $(2, 2, 2)$. Which has lower MSE? Which has lower mean absolute error? What does this reveal about the choice of loss?

Key idea. A loss function specifies which errors the fitting procedure should care about.

2. Fit a line with transparent small numbers

Least squares minimizes $S(b, w) = \sum_i (y_i - b - wx_i)^2$. At a minimum,

$$\frac{\partial S}{\partial b} = -2\sum_i (y_i - b - wx_i) = 0 \quad \Longrightarrow \quad b = \bar{y} - w\bar{x}.$$

Substitute this expression and differentiate with respect to w to obtain

$$\boxed{\widehat{w} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \widehat{b} = \bar{y} - \widehat{w}\bar{x}.}$$

The denominator must be positive. A constant feature cannot separately identify slope and intercept. The numerator measures how feature and target vary together; the denominator adjusts for the feature's spread.

Worked calculation

Take $x = (0, 1, 2, 3)$ and $y = (1, 2, 2, 4)$. Then $\bar{x} = 1.5$, $\bar{y} = 2.25$, the numerator is 4.5, and the denominator is 5. Thus $\widehat{w} = 0.9$ and $\widehat{b} = 0.9$.

x	0	1	2	3
y	1	2	2	4
$\widehat{y}$	0.9	1.8	2.7	3.6
r	0.1	0.2	−0.7	0.4

The residual sum of squares is 0.70. Predicting the mean 2.25 for every row gives 4.75. Fitting a slope improved training error, but these four observations do not establish future predictive performance.

```
x_bar, y_bar = x.mean(), y.mean()
w = ((x-x_bar)*(y-y_bar)).sum() / ((x-x_bar)**2).sum()
b = y_bar - w*x_bar
prediction = b + w*x
residual = y - prediction
```

The notebook checks agreement with `LinearRegression`. Agreement between implementations checks the calculation; it does not validate the task or dataset.

The constant baseline is itself fitted

For a constant predictor a, minimizing $\sum_i (y_i - a)^2$ gives $a = \bar{y}$. Use the training mean for validation and test predictions. Replacing it by the test mean uses outcomes unavailable to a new predictor and changes the comparison.

Check your understanding. If payload is converted from MB to KB, how should the slope change? Should predictions change? State the unit conversion you assume.

Key idea. Coefficients have units; interpret them through the representation used to fit the model.

3. Multiple features and array operations

Payload alone omits queue length. Extend the predictor to

$$\widehat{y} = b + w_s s + w_q q.$$

Now w_s describes a one-MB increase holding queue length fixed, and w_q is milliseconds per extra queued request holding payload fixed. These interpretations differ from one-feature slopes when the inputs are associated.

The notebook uses a separate simple simulator to make fitting transparent:

$$s \sim U(0.5, 10), \quad q \sim \text{Poisson}(3), \quad \epsilon \sim N(0, 8^2), \qquad y = 40 + 8s + 3q + \epsilon.$$

Features and noise are independent. The model contains the generating mean, but finite-sample estimates need not equal 40, 8, and 3. Noise also means that perfect predictions of new outcomes are not expected. Gaussian noise is a teaching approximation, not a physical nonnegativity guarantee for latency.

Put the intercept into the design matrix

$$D = \begin{pmatrix} 1 & s_1 & q_1 \\ \vdots & \vdots & \vdots \\ 1 & s_n & q_n \end{pmatrix}, \qquad \beta = \begin{pmatrix} b \\ w_s \\ w_q \end{pmatrix}, \qquad \widehat{y} = D\beta.$$

D has shape $(n, 3)$ and β shape $(3,)$. Use a target of shape $(n,)$. Subtracting an $(n, 1)$ target from an $(n,)$ prediction can silently create an (n, n) array through broadcasting, producing a completely different loss.

```
D = np.column_stack([np.ones(len(X_train)), X_train])
beta = np.linalg.lstsq(D, y_train, rcond=None)[0]
prediction = D @ beta
assert prediction.shape == y_train.shape
```

Explicitly forming and inverting $D^T D$ is unnecessary and can worsen numerical problems. If two features are duplicates, their separate coefficients are not uniquely identified, even when their combined contribution can still be predicted.

Scaling changes coordinates

Use $z_j = (x_j - \mu_j)/a_j$ with training means and scales. An unpenalized model with an intercept has the same prediction family under invertible scaling. A scaled slope measures change per training standard deviation. Recover original units with

$$w_j^{\text{original}} = w_j^{\text{scaled}}/a_j, \qquad b^{\text{original}} = b^{\text{scaled}} - \sum_j w_j^{\text{original}} \mu_j.$$

Check your understanding. Why can two models have similar predictions but different coefficients when two features are almost duplicates? What information is missing?

Key idea. Array shapes and feature definitions are part of the mathematical model.

4. Gradient descent: learn by correcting errors

A direct solver is convenient for least squares. Gradient descent supplies a general learning pattern that also applies to logistic regression and neural networks. Use half MSE to simplify the derivative:

$$J(\beta) = \frac{1}{2n}\|D\beta - y\|^2, \qquad \nabla J(\beta) = \frac{1}{n}D^\mathsf{T}(D\beta - y).$$

Each gradient component describes the local change in loss as one coefficient increases. Multiplication by D^T sums errors weighted by each feature. The intercept column of ones gives the average prediction-minus-target error.

$$\boxed{\beta \leftarrow \beta - \eta\nabla J(\beta).}$$

The learning rate $\eta > 0$ controls step size. A sufficiently small step opposite a nonzero gradient locally reduces loss. A large step can cross the minimum and increase loss. Scaling can make one step size work better across coefficient directions.

One step by hand

Use $(x, y) = (0, 1), (1, 3)$ and start at $(b, w) = (0, 0)$. Prediction-minus-target errors are $(-1, -3)$, so

$$\nabla J = (-2, -1.5), \qquad \eta = 0.1 \Longrightarrow \beta_{\text{new}} = (0.2, 0.15).$$

Initial half MSE is $(1 + 9)/4 = 2.5$. New predictions are $(0.2, 0.35)$, giving $(0.8^2 + 2.65^2)/4 \approx 1.916$. This is progress, not the final fitted line.

```
beta = np.zeros(D.shape[1])
for step in range(max_steps):
    error = D @ beta - y_train
    gradient = D.T @ error / len(y_train)
    if np.linalg.norm(gradient) < tolerance:
        break
    beta -= learning_rate * gradient
```

Record the loss and whether the stopping criterion was reached. A loop ending at its iteration cap is not automatically converged. A nearly flat loss curve can also reflect a very small step rather than proximity to the optimum.

The notebook compares the analytic gradient with central finite differences, then checks gradient-descent predictions against the direct solver. These check the implementation. Validation error answers the separate question of whether the prediction relationship works on unseen rows.

Check your understanding. Removing the factor $1/2$ from the loss changes the gradient by what factor? What learning-rate adjustment preserves the same update?

Key idea. Fitting reduces an objective; evaluation is a separate experiment.

5. Generalization and residual diagnostics

For m held-out observations, report errors in the target's units:

$$\mathrm{MAE} = \frac{1}{m}\sum_i |y_i - \widehat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_i (y_i - \widehat{y}_i)^2}.$$

Both are in milliseconds. RMSE emphasizes large misses more strongly. An elaborate model can perform worse than the training-mean baseline on new data even if its training loss is smaller.

A small, explicit selection protocol

Reserve 60/20/20 partitions. Fit a payload-only line and a payload-plus-queue model on the same training rows. Compare validation RMSE and choose its lower value. Freeze the representation and fitted model, then report test MAE and RMSE beside the baseline. No further changes follow the test table.

For example, validation residuals $(1, -3)$ give RMSE $\sqrt{5} \approx 2.236$ for model A; residuals $(2, -2)$ give 2 for B. A predeclared RMSE rule selects B. Both have MAE 2, illustrating why the selection metric should be chosen before results.

A residual plot can expose missing structure

Plotting training residuals against queue may show increasing errors when queue is omitted. Curvature suggests a missing nonlinear relationship. Changing residual spread suggests outcome variability changes with input. These are clues, not proofs of causes or guarantees of future performance.

A flexible polynomial may reduce training residuals yet behave unstably between observations. Added complexity must be justified on held-out data. Conversely, a simple line can underfit a nonlinear relationship even after optimization converges. More optimizer steps cannot repair a missing feature or an unsuitable model family.

Interpret a score with its limits

Suppose test RMSE is 9 ms against a 25 ms baseline. The selected model improves squared-error prediction on these held-out simulated requests. That does not mean every individual error is below 9 ms, that the coefficients are causal, or that another service will achieve the same error.

Finite validation sets also create variability in model rankings. If several candidates have nearly identical errors, small sample changes may change the winner. Report the observed comparison instead of declaring a universal ordering of algorithms.

Check your understanding. Training loss falls while validation RMSE rises. What does each observation say about optimization and prediction? How could a larger independent sample help?

Key idea. A useful model improves the relevant held-out baseline in the intended setting.

6. Closing exercise: from formula to experiment

Use $x = (0, 1, 2)$ and $y = (1, 2, 2)$.

1. Compute both means, the least-squares slope, and the intercept.
2. List predictions and residuals. Calculate MSE and RMSE.
3. Compute the training-mean baseline's MSE and compare with the line.
4. With half MSE, start at $(b, w) = (0, 0)$ and take one gradient step of size 0.1. Show both gradient components.
5. Why does the first update differ from the exact solution? Give a stopping criterion and an independent implementation check.
6. A fourth input is $x = 10$. Why is this extrapolation, and why does low training error not establish its reliability?

Application: latency from two inputs

The student notebook supplies 600 independent requests with

$$y = 40 + 8\,\text{payload} + 3\,\text{queue} + \epsilon, \qquad \epsilon \sim N(0, 8^2).$$

Fit a training-mean baseline and a two-feature linear model. Complete one vectorized gradient step in standardized coordinates. Evaluate the fixed linear model on test rows using MAE and RMSE. Preserve the supplied data, seeds, and partitions.

Interpret both coefficients in original units. Explain why their fitted values need not match the simulator exactly, and why correct model form does not imply zero test error. State whether the library fit and your gradient calculation solve the same objective, including any normalization factor.

Optional extension: learning-rate sensitivity

On training data, compare rates 0.01, 0.1, and 3.0 after scaling. Plot half MSE against iteration. Set a maximum number of updates and stop if values become nonfinite. Explain slow progress, convergence, and divergence. Do not select a prediction model by repeatedly comparing test errors during these exploratory runs.

Submission. Completed notebook with saved outputs, hand calculations, coefficient interpretations, and a paragraph distinguishing optimization checks from evidence about prediction. The separate PDF brief gives the marking guide.

Connection to session 3. A linear score can also support classification. We will change the output interpretation and loss, then separate estimating an event probability from deciding when to raise an alarm.

Key idea. Report the fitted rule, the objective, and evidence on data that did not fit it.